G078 문희주

AI 서비스 Mini Project — 과제 2 (신규 AI Agent) 최종 보고서

WISH FINDER

자막 기반 영상 장면 찾기 AI Agent (첫 적용 대상: NCT WISH 영상 3 편)

한 줄 요약 정확한 대사나 영상 제목은 기억나지 않지만 "어떤 장면이었는지"만 어렴풋이 기억하는 팬이, 한국어로 장면을 설명하면 영상의 정확한 재생 시점을 찾아 주는 **근거 기반 검색 Agent** 이다. 자막에서 후보를 찾고, 원본 자막과 다시 대조해 검증한 뒤, 검증을 통과한 장면만 영상 링크와 함께 보여 준다.

결과 한눈에

- **동작**: 브라우저에서 질문 입력 → 진행 단계 표시 → 검증된 장면 카드 → 해당 시점 영상 재생까지 실제로 동작한다.
- **정확도**: 정답 평가셋의 장면 질문 18 개 중 Agent 가 1 위로 정답을 맞힌 것이 **16 개(88.9%)**이다. 같은 조건의 일반 검색(Hybrid)은 11~15 개였다. 다만 차이가 1~5 문항이고 질문당 1 회만 실행해서, Agent 가 더 낫다고 단정할 수는 없다.
- **안전성**: 근거 없이 화자를 단정한 경우 **0 건**, 자막으로 알 수 없는 요청 2 개를 모두 거부했다.
- **개선 전/후**: 1 위 적중은 16/18 로 같았고, 화자 질문 처리가 2/4 → 3/4 로 나아졌으며 LLM 호출이 16% 줄었다(50 → 42 회). 대신 입력 토큰이 24% 늘었다.
- **한계**: Multi-Agent(여러 Agent 의 협업)는 구현하지 않았다. 같은 의미의 질문도 표현에 따라 찾기도 하고 못 찾기도 하며, 후속 질문에서 틀린 장면을 내는 사례가 있다.

저장소: https://github.com/creepygrape/skala-AICapstone-mini-2 (실행 방법은 부록 B)

측정한 값과 측정하지 못한 값을 구분해서 적었다. 노란 표시는 제출 전에 채울 부분이다.

읽기 전에: 용어 설명

표 1. 이 보고서에서 쓰는 용어

용어	뜻
cue(자막 한 줄)	자막 파일(SRT)의 한 항목. 시작·종료 시각과 텍스트로 이루어진다. 이 서비스는 cue 를 유일한 근거로 쓴다.
A / B / C	비교하는 세 방식. A = BM25 키워드 검색, B = Hybrid 검색(키워드 + 의미), C = Hybrid 검색 + Agent(검증·재검색).
v2 / v3	개선 전 / 개선 후 버전(v3: 화자 질문 판정, 번역 자막 대응, 중복 재검색 제거 등 적용).
결과 상태	찾음(found) / 못 찾음(not_found) / 애매함(ambiguous) / 자막으로 알 수 없는 요청(unsupported).
정답 평가셋(gold)	사람이 영상을 확인해 정답 시점을 정해 둔 평가용 질문 20 개(장면 질문 18, 자막으로 알 수 없는 요청 2).
1 위 적중 / 상위 3 개 적중	검색 1 위 결과가 정답 시점(±10 초)인 비율 / 상위 3 개 안에 정답이 있는 비율.

1. 진행 사항 소개

1.1 과제 주제와 문제 정의

사용자 문제. 팬은 "마니또를 뽑다가 자기 자신을 뽑은 장면"처럼 상황만 기억한다. 영상 제목이나 정확한 대사를 모르면 일반 키워드 검색으로는 찾기 어렵고, 1 시간짜리 영상을 직접 돌려 보는 데도 시간이 든다.

해결 방식. 한국어로 장면을 설명하면 자막에서 후보를 찾고, 원본 자막과 대조해 검증한 뒤 영상 주소와 시작 시점을 알려 준다. 화면에는 해당 시점의 영상과 앞뒤 자막을 함께 보여 준다.

> **기술적 제약(사용자 문제와 구분).** 실제 영상에는 한국어·일본어 화면 자막이 있지만, 확보할 수 있는 추출 자막은 **영어 번역본**이고 **화자(말한 사람) 표시가 없다**. 그래서 이 서비스는 음성과 영상을 이해하지 않고 자막 텍스트(대사와 괄호 속 화면 편집 설명)만 쓰며, 말한 사람은 단정하지 않는다. 예를 들어 "(RIKU chose RIKU)"는 화자 표시가 아니라 화면 편집 설명이다.

1.2 주요 task 와 결과

표 2. 주요 task 와 결과

영역	한 일	결과
데이터	자막 파일을 읽어 cue 로 저장하고, 영상별로 20 초 단위로 묶어 검색용 단위(청크)를 만들었다(영상 간 혼합 없음).	cue 3,750 개 → 청크 869 개, 영상 3 편
검색(RAG)	키워드 검색(BM25)과 다국어 의미 검색(bge-m3)을 합친 Hybrid 검색을 만들었다.	A(BM25)·B(Hybrid) 기준선 구현
Agent	LangGraph 로 분류 → 검색 → 검증 → (재검색) → 결과 정리 흐름을 구성했다.	재검색 최대 2 번, 시간 제한, 오류 시 폴백
MCP	자막 조회 도구 4 개를 MCP 서버로 만들고, 그래프가 실제로 호출한다.	읽기 전용 도구 4 개
웹/API	FastAPI 와 Vue 로 대화형 화면을 만들고, 진행 단계를 실시간(SSE)으로 보여 준다. 검색 기록, 앞뒤 자막, 후속 질문을 지원한다.	브라우저에서 end-to-end 동작
운영	LLM 백엔드를 환경변수로 교체(Ollama 개발 / OpenAI 제출)하고, LangSmith 로 호출·토큰·지연을 기록한다.	LangSmith 에서 요청별 토큰·지연 확인
평가·문서	A/B/C 평가 도구와 설계 결정 기록(ADR 11 건)을 만들었다.	3 장, 부록 C

1.3 대상 범위와 데이터 원칙

- 구현·데이터·평가는 **NCT WISH 영상 3 편**으로 한정했다. 대상이 바뀌어도 코드를 고치지 않도록 대상별 정보(자막 언어, 번역 여부, 이름 사전)는 별도 설정 파일에 두었다. 다른 대상을 실제로 지원한다고 주장하지는 않는다.
- 제삼자의 원본 자막은 공개 저장소에 올리지 않는다. 화면에는 영상 링크와 짧은 인용만 보여 준다.
- **인용문, 시각, 영상 주소는 LLM 이 쓰지 않고, 코드가 원본 cue 에서 만든다.** 자막에 이름이 나온 것(언급된 멤버)과 그 말을 한 사람(화자)은 구분한다.

2. 사용한 기술 요소

2.1 전체 구조

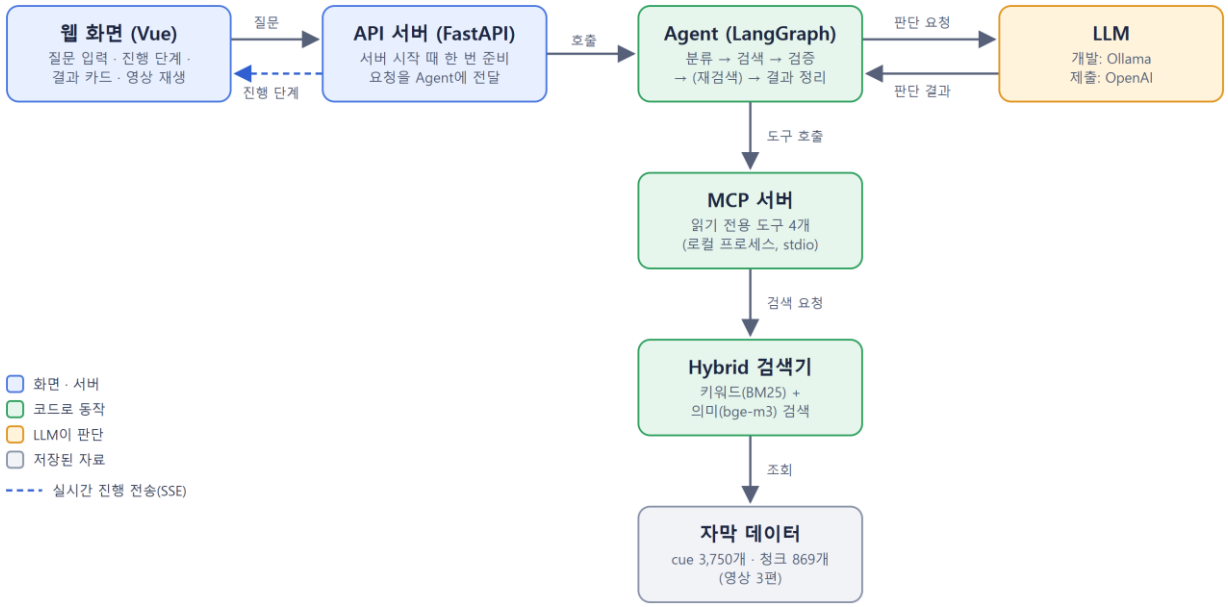


그림 1. 시스템 구성. 서버가 시작될 때 Agent, MCP 연결, 검색 인덱스를 한 번만 준비하고 요청마다 다시 만들지 않는다.

2.2 LangGraph Agent (단일 Agent 워크플로)

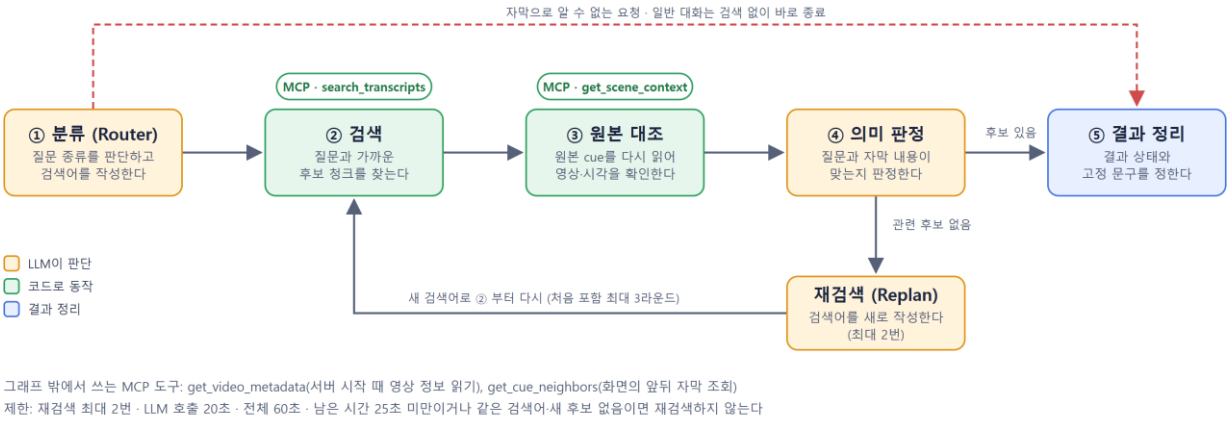


그림 2. LangGraph 실행 흐름. 다음 단계는 코드의 조건문이 정하고, LLM 은 세 곳(분류, 판정, 검색어 재작성)에서만 판단한다. 검색(②)은 search_transcripts, 원본 대조(③)는 get_scene_context 를 호출한다.

표 3. Agent 단계별 역할

단계	하는 일
Router	질문을 분류(장면 검색 / 화자 질문 / 자막으로 알 수 없는 요청 / 일반 대화)하고, 자막 언어(대상 설정에서 정하며 지금은 영어)로 검색어를 만든다. 검색이 필요 없으면 바로 종료한다.
Search, Context	후보를 검색하고, 후보마다 원본 cue 를 다시 읽어 영상, 시각, cue 포함 여부를 **코드로** 검증한다. 검증에 실패한 후보는 버린다.
Verification	원래 한국어 질문과 원본 자막(앞뒤 문맥 포함)을 비교해 관련 있음 / 불확실 / 관련 없음으로 판정한다.
Replan	관련 후보가 없고 남은 시간이 25 초 이상일 때만, 핵심 명사를 바꿔 검색어를 다시 작성한다. 최대 2 번(처음 검색 포함 3 라운드)이며, 같은 검색어이거나 새 후보가 없으면 멈춘다.
Finalize	결과 상태와 고정 설명 문구를 정한다. 모델이 쓴 자유 문장은 사용자에게 보이지 않는다.

- **오류 처리**: 모델이나 도구의 오류는 재검색 사유로 쓰지 않는다. 검증 전 후보만 있으면 "부분 결과"와 경고로, 이전에 불확실 판정이 있었으면 "애매함"으로 응답하고, 아무것도 없으면 안전한 오류 코드만 보낸다. 검증 전 결과는 화면에 보내지 않는다.

2.3 MCP (Model Context Protocol)

표 4. Agent 가 쓰는 MCP 도구 2 개 (모두 읽기 전용, 파일 경로를 인자로 받지 않음)

도구	하는 일	쓰이는 곳
search_transcripts	질문으로 후보 청크를 검색	검색 단계(질문마다). 계산이 무거워 동시에 1 건만 실행
get_scene_context	영상의 특정 시각 주변 **원본 cue** 조회	원본 대조 단계(후보마다). 영상 경계 안에서만 반환

표 5. 서버·화면이 쓰는 MCP 도구 2 개 (Agent 는 쓰지 않음)

도구	하는 일	쓰이는 곳
get_video_metadata	영상 제목·주소·확인 상태 조회	API 서버가 시작할 때 한 번 호출해 메모리에 저장
get_cue_neighbors	결과 cue 의 앞뒤 cue 조회	화면의 앞뒤 자막(사용자가 시점을 선택할 때). API 서버가 호출

- 공식 Python SDK 2.x 의 MCPServer 를 쓰고, 로컬 프로세스(stdio)로 실행한다. 연결은 서버 시작 때 한 번 열어 모든 요청이 공유한다.

- **설계 선택과 대가**: 자막 데이터는 MCP 서버만 읽는 단일 소유자 구조라서 API 서버도 영상 정보와 앞뒤 자막을 MCP 를 거쳐 조회한다. 그 대가로 Agent 용이 아닌 화면용 조회가 MCP 도구 계약에 섞인다.
- **도구를 나눈 이유**: ① 입력이 다르다(질문 / 영상과 시각 / cue ID). ② 무게가 다르다. 검색만 계산이 무거워 동시에 1 건으로 제한하고, 나머지는 가벼워 검색 중에도 호출할 수 있다. ③ 검색 결과의 청크 텍스트는 검색용으로 가공(중복 제거, 이어 붙이기)한 것이라, 인용과 시각은 get_scene_context 가 돌려주는 **가공하지 않은 원본 cue** 로 다시 확인한다. 청크 만들기에서 생긴 오류가 인용과 시각으로 번지지 않게 하기 위해서이다. ④ 파일 경로를 인자로 받지 않아 LLM 이나 코드가 파일을 직접 다루지 못하게 하기 위해서이다. 앞뒤 자막 조회(get_cue_neighbors)는 화면을 만들면서 나중에 추가했고, 이미 쓰고 있는 도구의 입출력을 바꾸지 않으려고 따로 두었다.

2.4 RAG: Hybrid 검색과 원본 자막 근거

- **청킹**: 영상별로 20 초 단위를 10 초씩 겹쳐 만든다. 영상을 가로질러 묶지 않으며, 각 청크는 포함된 cue 목록을 가지고 있어 결과를 항상 원본 cue 로 되돌릴 수 있다. 자막 한 줄은 1~3 초로 짧아 한 줄만으로는 의미 검색이 어렵고, 한 장면은 보통 수십 초에 걸쳐 이어진다고 보아 20 초로 묶었다. 10 초(절반)씩 겹치는 것은 장면이 단위의 경계에 걸쳐도 어느 한 단위에는 온전히 들어가게 하기 위해서이다. **이 값은 설계 때 정한 가정이며, 다른 길이와 비교하는 실험은 하지 않았다.**
- **검색**: 키워드 검색(BM25) 상위 20 개와 의미 검색(bge-m3, 다국어) 상위 20 개를 RRF 로 합친다. RRF 는 각 목록에서 순위가 r 인 후보에 **1 / (60 + r)** 점수를 주고 두 목록의 점수를 더하는 방식이며, 60 은 기본값을 사용했다. 합친 뒤 같은 영상에서 10 초 이내인 후보는 하나만 남긴다. 청크 안에서 질문과 가장 가까운 cue 를 대표 근거로 고른다.
- **질문 언어와 자막 언어가 다를 때**: 이 프로젝트는 한국어 질문에 영어 번역 자막이다. Router 가 핵심 명사의 동의어를 함께 넣어 **자막 언어로 검색어를 작성**하고, 판정 단계에서도 번역 때문에 단어가 달라도 뜻이 같으면 인정한다.
- **이중 첫 검색(선택 기능, 기본 꺼짐)**: 첫 검색에서 Router 가 자막 언어로 작성한 검색어와, 사용자가 입력한 질문 그대로(입력한 언어 그대로)를 함께 검색해 번역 어휘 차이를 보완한다(3.4 절).

2.5 의존성 주입

- FinderAgent 는 필요한 것(MCP 연결, 영상 정보, 모델, 도구 호출 함수, 로그 설정 등)을 모두 **생성자로 받는다**. 운영에서는 서버가 실제 연결과 모델을 넣고, 테스트에서는 가짜 모델과 가짜 연결로 바꿔 넣는다.
- **LLM 교체**: 환경변수 하나(WISH_LLM_BACKEND)로 Ollama(개발, 무료)와 OpenAI(시연·제출, 과금)를 바꾼다. 프롬프트와 출력 형식은 공통 부분에 한 벌만 두고 각 백엔드는 전송만 맡아서, 그래프·MCP·검색·테스트 코드는 바뀌지 않는다.

- 두 백엔드의 호환성 테스트는 가짜 HTTP 응답으로 하므로 **과금되지 않는다**.

2.6 LangSmith 와 관측성

- **선택 기능**: WISH_LANGSMITH_ENABLED=1 과 LANGSMITH_API_KEY 가 모두 있을 때만 켜진다. 켜면 그래프 단계와 **LLM 호출마다** 모델명, 입출력 토큰, 지연이 하나의 트레이스로 올라가 호출별 비용을 볼 수 있다.
- 질문과 후보 자막이 함께 올라가므로 개인 비공개 프로젝트에서만 쓴다. 키가 없어도 핵심 기능은 동작하며 같은 내용이 로컬 로그 파일(logs/agent.jsonl)에도 남는다.
- 평가를 두 번(v2, v3) 실행한 결과가 LangSmith 프로젝트 wish-finder-eval 에 올라가 있다. 개요 화면(그림 3)은 **트레이스 160 개, 모델 호출 188 회, 오류율 0%, 토큰 135.5K, 비용 $0.30** 을 보여 준다. 이 값은 로컬 기록과 일치한다(호출 98 + 90 = 188 회, 토큰 61,066 + 74,438 = 135,504). 비용은 LangSmith 가 모델명과 토큰으로 계산한 값이라 실제 청구액과 다를 수 있다.

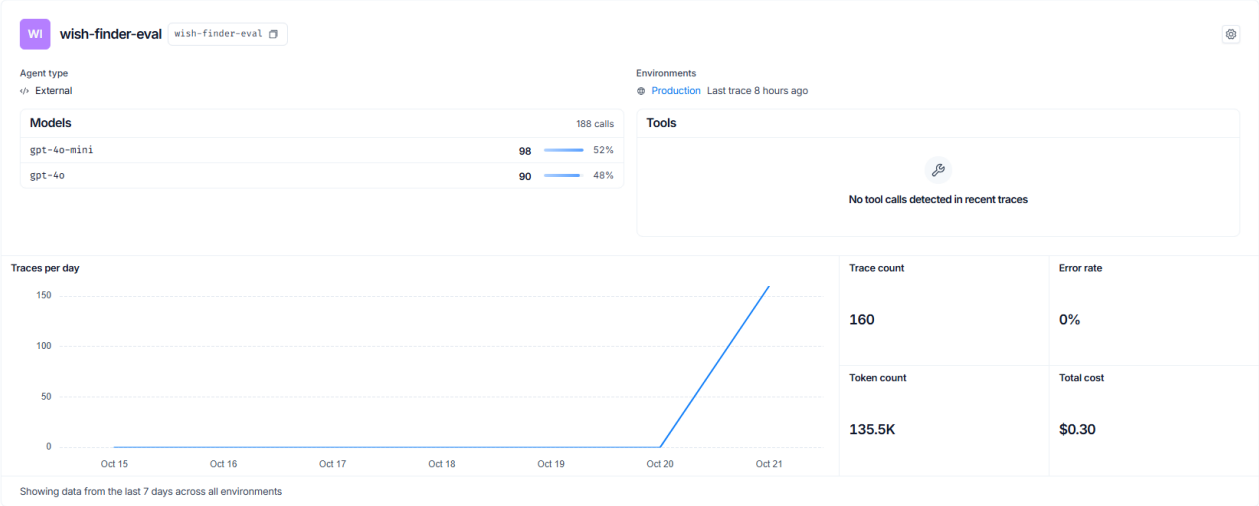


그림 3. LangSmith 프로젝트 wish-finder-eval 개요 (최근 7 일, v2·v3 평가의 합계). 모델별 호출 수, 트레이스 160 개, 오류율 0%, 토큰 135.5K, 비용 $0.30.

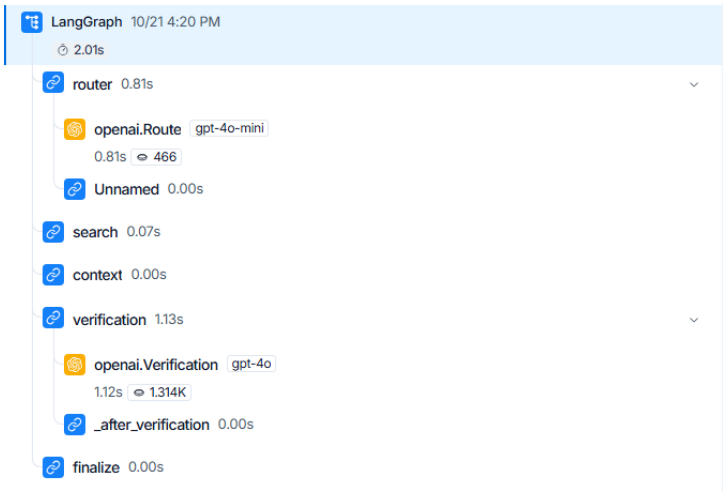


그림 4. LangSmith 트레이스 트리 (재검색 없이 끝난 평가 문항 1 건, 질문은 비공개라 트리만 표시). 전체 2.01 초 = router 0.81 초(LLM, gpt-4o-mini, 466 토큰) + search 0.07 초 + context 0.00 초 + verification 1.13 초(LLM, gpt-4o, 1.314K 토큰) + finalize. 단계 이름이 코드의 그래프와 그대로 대응한다.

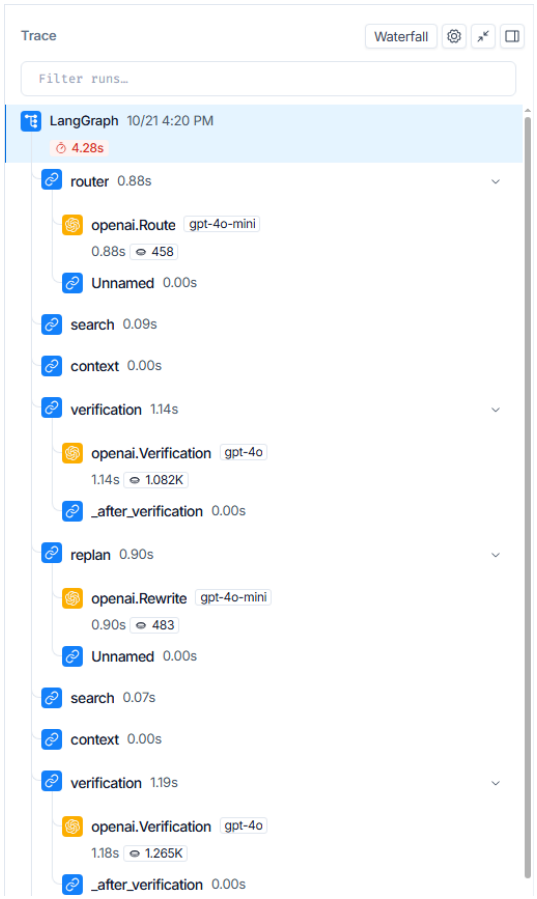


그림 5. LangSmith 트레이스 (재검색 1 번, 2 라운드). 전체 4.28 초 = router 0.88 초 + 1 라운드(search 0.09 초, verification 1.14 초) + replan 0.90 초(LLM, gpt-4o-mini, 483 토큰) + 2 라운드(search 0.07 초, verification 1.19 초). 맨 아래 finalize 는 화면 밖으로 이어진다. 위 그림의 재검색 없는 건(2.01 초, LLM 2 회, 약 1.8K 토큰)과 비교하면 재검색 1 번이 지연, LLM 호출(2 → 4 회), 토큰(약 3.3K)을 대략 두 배로 늘린다. 서로 다른 질문을 1 번씩 본 것이라 통제된 비교는 아니다.

2.7 모듈화

표 6. 모듈 구성

디렉터리	책임
backend/app/main.py, sse.py	HTTP, 실시간 진행 전송(SSE), 서버 시작·종료 시 자원 관리, 오류를 사용자용 메시지로 변환
backend/app/agent/	LangGraph 그래프, LLM 공통 규칙과 Ollama·OpenAI 연결, 로그·LangSmith 기록
backend/app/mcp/	MCP 서버와 클라이언트, 동시 검색 제한
backend/app/retrieval/	자막 파서, 청크 생성, Hybrid 검색
backend/app/services/	도구 호출, 원본 대조 검증, Agent 없는 검색(A/B), 영상 정보
frontend/src/	Vue 화면, 실시간 이벤트 처리, YouTube 플레이어, 검색 기록 저장
eval/, scripts/	A/B/C 평가 도구(정답 평가셋 승인 확인 포함), 실행 스크립트

2.8 단위·통합 테스트

- **백엔드 177 개 통과, 1 개 건너뜀**(Windows 에서 권한이 없으면 건너뛰는 파일 링크 테스트), **프런트 43 개 통과**, 프런트 빌드 성공.
- 모델 호출은 가짜 모델과 가짜 HTTP 응답으로 대신해서 테스트가 **과금되지 않고 결과가 항상 같다**. 실제 MCP 서버(stdio)를 띄우는 테스트와, 합성 자막으로 MCP·LangGraph·API 를 연결하는 통합 테스트도 포함한다.

표 7. 백엔드 테스트 영역별 내용 (합계 178 개: 177 개 통과, 1 개 건너뜀)

영역	검증 내용	개수
Agent 그래프	경로와 재검색 횟수 상한(0/1/2 회), 같은 검색어·중복 후보에서 종료, 모델·도구 오류 시 폴백, 후속 질문과 이미 본 장면 제외, 이중 첫 검색	55
LLM 교체	Ollama/OpenAI 호환성(가짜 HTTP), LangSmith 로 보내는 내용(가짜 세션)	19
자막·검색	원본 cue·시각·영상 경계, 괄호 속 편집 설명, 청크 만들기와 청크 길이 설정, 도구 호출과 근거 검증	37
설정·API·실시간 전송	대상 프로필, JSON API 와 앞뒤 자막 조회, 실시간 이벤트의 순서·분할 수신·취소와 자원 정리	33
평가 도구	Agent 평가 경로, 정답 평가셋 승인 해시, 비공개 경계, 지표 계산	32
실제 MCP 통신	실제 stdio MCP 서버를 띄워 호출하고 종료	2

- **프런트 43 개**: 결과 카드(영상별 묶음, 선택 표시)·진행 단계·상세 패널 13 개, 플레이어 수명과 정지 13 개, 실시간 이벤트 분할 수신 7 개, 진행 단계 계산 5 개, 검색 기록 저장·복원 5 개.

- **테스트가 다루지 못하는 것**: LLM 의 실제 응답 품질은 가짜 모델로 확인할 수 없어 3 장의 평가로 확인한다. 화면의 클릭 동작은 자동 테스트가 아니라 Chrome 에서 직접 확인했고, 브라우저 자동 점검 스크립트는 실행하지 않았다.

2.9 구현한 기능과 최종 실행 화면

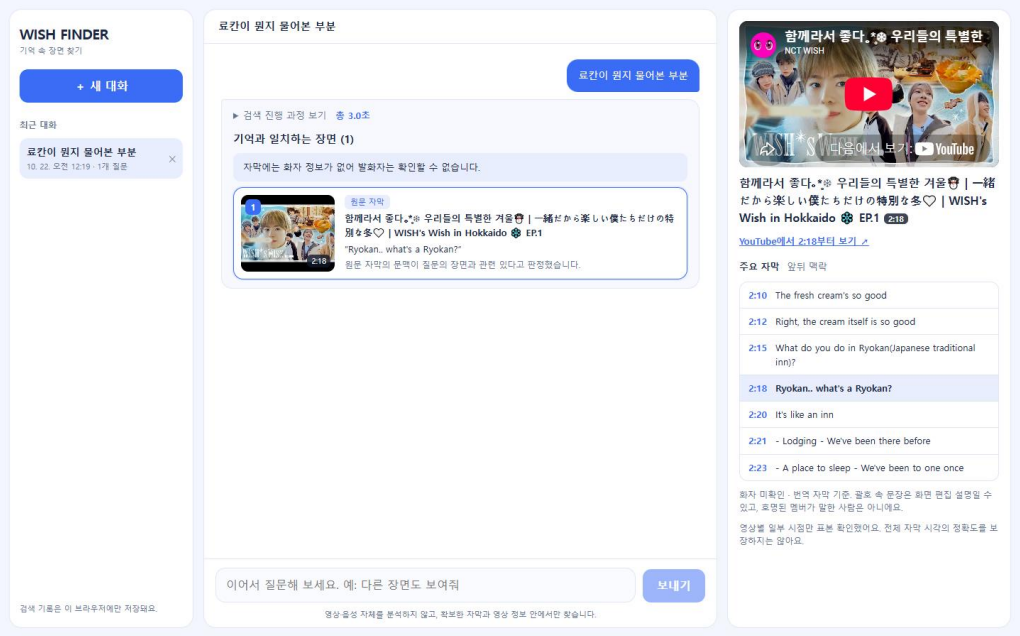


그림 6. 결과 화면. 검증을 통과한 장면 카드(원본 자막 인용), 선택한 장면의 영상, 원본 앞뒤 자막(강조된 줄이 근거). 화자는 "확인할 수 없음"으로 안내한다.

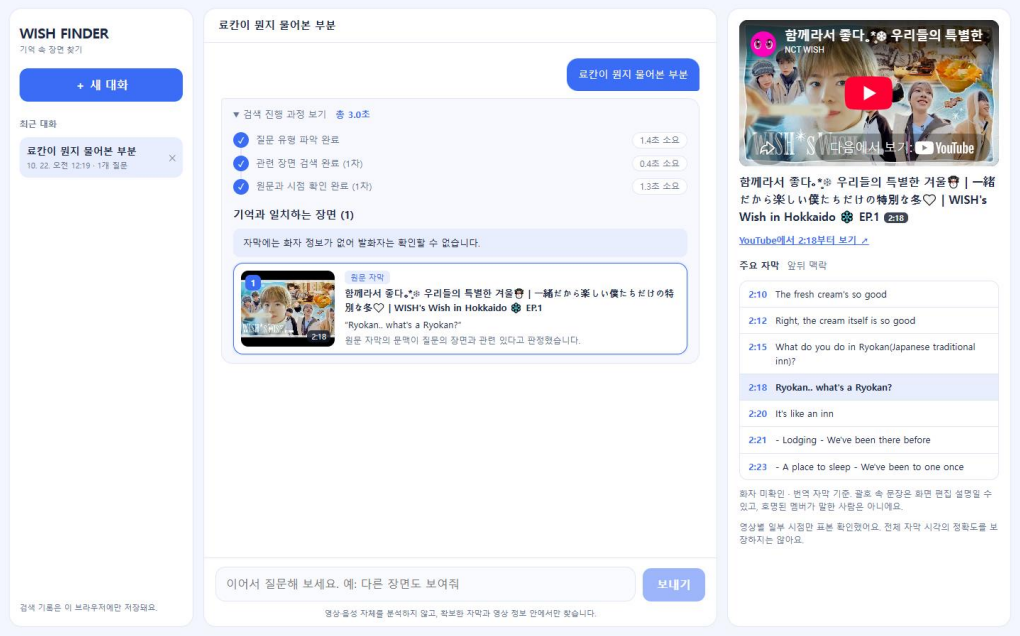


그림 7. Agent 진행 단계: 질문 유형 파악 1.4 초 → 장면 검색 0.4 초 → 원문·시점 확인 1.3 초, 총 3.0 초. 검증 전 결과는 화면에 나타나지 않는다.

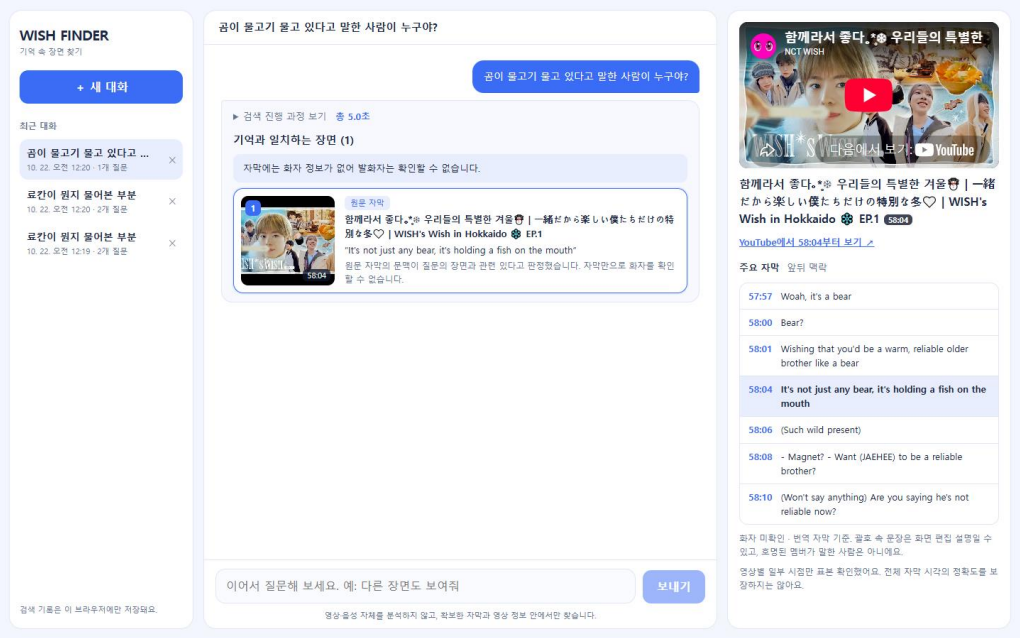


그림 8. 화자 질문("곰이 물고기 물고 있다고 말한 사람이 누구야?"). 장면(58:04)은 찾되 "자막만으로 화자를 확인할 수 없습니다"라고 안내하고 말한 사람을 단정하지 않는다(총 5.0 초).

3. Before/After 실측 비교 (보너스 항목)

결론부터. 개선 후(v3)에도 1 위 적중 수는 16/18 로 같았다. 달라진 것은 화자 질문 처리(2/4 → 3/4), LLM 호출 수(50 → 42 회), 평균 지연(2.88 → 2.57 초)이고, 입력 토큰은 오히려 늘었다. 아래 모든 수치는 **질문당 1 회 실행**한 값이므로 1~2 문항 차이는 우연일 수 있다.

3.1 측정 조건

- **같은 입력**: 정답 평가셋(gold) 20 문항. 장면 질문 18 개(일반 13, 같은 말을 하는 장면이 여러 개인 질문 1, 화자 질문 4)와 자막으로 알 수 없는 요청 2 개. 같은 영상 3 편, 같은 정답. 정답은 영상이 맞고 링크 시각이 정답 구간 ±10 초 안일 때이다.
- **LLM**: OpenAI. 분류와 검색어 재작성은 gpt-4o-mini, 판정은 gpt-4o(temperature 0, seed 42). 시연과 제출은 OpenAI 기준이라 이 장의 수치는 모두 OpenAI 로 측정한 값이다.
- **개선 전(v2) / 후(v3)**: v3 는 v2 에서 드러난 문제 네 가지를 고친 버전이다. 같은 평가셋과 같은 명령으로 다시 측정했다.
- 고친 점 ① **화자 질문의 판정 기준**: "누가 말했어?"라고 물어도 자막에는 말한 사람이 적혀 있지 않다. v2 는 이 때문에 장면이 맞아도 "불확실"로 판정하는 일이 있었다. v3 는 장면이 맞으면 "관련 있음"으로 판정하고, 말한 사람은 "확인 불가"로 따로 안내한다.
- 고친 점 ② **번역 자막 대응**: 자막이 영어 번역본이라 한국어 질문의 단어와 자막의 단어가 다를 수 있다. 번역 때문에 단어가 달라도 뜻이 같으면 인정하도록 판정 지침을 바꾸고, 검색어에는 핵심 명사의

동의어를 함께 넣게 했다. 자막에 자주 나오는 멤버 이름의 표기는 설정 파일의 이름 사전으로 알려 준다.

- 고친 점 ③ **중복 재검색 제거**: 재검색이 이미 판정한 후보를 다시 판정하거나 같은 검색어를 반복해 시간과 LLM 호출만 쓰는 일이 있었다. v3 는 이미 판정한 후보는 다시 묻지 않고, 새 후보가 없으면 재검색을 멈춘다.
- 고친 점 ④ **거절된 후보를 보여 주며 검색어 재작성**: 재검색에서 검색어를 다시 작성할 때 직전에 "관련 없음"으로 판정된 후보의 인용문을 함께 보여 준다. 같은 계열의 말을 또 찾지 않고 핵심 명사를 바꾸게 하기 위해서이다.

첫 검색어 조건. 방식마다 처음 쓰는 검색어가 달라서(A·B 는 질문 원문, C 는 Agent 가 영어로 작성) 그대로 비교하면 "검색어 차이"와 "Agent 효과"가 섞인다. 그래서 세 방식이 같은 첫 검색어로 시작하는 두 조건을 모두 측정했다.

표 8. 첫 검색어 비교 조건 (재검색 검색어는 Agent 만 새로 작성한다)

조건	세 방식(A·B·C)이 모두 첫 검색어로 쓰는 것	이 장에서의 표기
조건 ① 질문 원문	사용자가 입력한 한국어 질문 그대로	Agent 는 C-ko
조건 ② 영어 검색어	Agent(Router)가 질문을 보고 자막 언어(영어)로 작성한 검색어	Agent 는 C

3.2 개선 전/후 (v2 → v3)

표 9. 개선 전/후 비교 (OpenAI, 같은 평가셋·같은 모델, 질문당 1 회 실행)

지표 (괄호는 전체 문항 수)	개선 전 (v2)	개선 후 (v3)	변화
Agent 1 위 적중 수 (장면 질문 18 개)	16	16	같음
Agent 상위 3 개 적중 수 (18 개)	16	16	같음
Agent 결과 상태 (20 개)	찾음 16, 애매함 1, 못 찾음 1, 알 수 없는 요청 2	찾음 17, 못 찾음 1, 알 수 없는 요청 2	애매함 1 → 찾음
화자 질문 처리 성공 수 (4 개)	2	3	+1
자막으로 알 수 없는 요청 거부 수 (2 개)	2	2	같음
틀린 장면을 "찾음"으로 답한 수	0	1	+1 (비슷한 장면이 여럿인 질문)
B(Hybrid) 1 위 적중 수, 조건 ② (18 개)	13	15	+2
A(BM25) 1 위 적중 수, 조건 ② (18 개)	5	13	+8
Agent LLM 호출 수 (20 문항 전체)	50	42	−16%
Agent 평균 / 최대 지연 (초)	2.88 / 6.26	2.57 / 6.74	평균 −11%

지표 (괄호는 전체 문항 수)	개선 전 (v2)	개선 후 (v3)	변화
Agent 입력 / 출력 토큰	29,351 / 2,090	33,753 / 1,717	입력 +15%, 출력 −18%
C 와 C-ko 합계 LLM 호출 / 토큰(입력·출력)	98 / 56,941·4,125	90 / 70,770·3,668	호출 −8%, 입력 **+24%**

- **재검색 횟수**: 화자 질문 1 건이 3 라운드(LLM 6 회)에서 1 라운드(LLM 2 회)로, 재검색으로 찾던 질문 2 건이 2 라운드에서 1 라운드로 줄었다. 재검색 1 번의 비용은 2.6 절 그림 5 에서 볼 수 있다(재검색 없는 건 2.01 초·LLM 2 회, 재검색 1 번인 건 4.28 초·LLM 4 회).
- **1 위 적중 수는 같다.** 한 문항을 얻고(화자 질문) 한 문항을 잃었다(같은 말을 하는 장면이 여럿인 질문에서 다른 비슷한 장면을 정답으로 냄). 확인된 이득은 화자 질문 처리, 재검색 효율, 키워드 검색(A/B)의 검색어 품질이다.
- A 의 1 위 적중이 5 에서 13 으로 오른 것은 Router 가 핵심 명사의 동의어를 함께 넣도록 프롬프트를 바꾼 효과이다(고친 점 ②). Agent 의 재검색이나 판정과는 관계가 없다.

3.3 A / B / C 비교 (개선 후)

표 10. 1 위 적중 수 (장면 질문 18 개 중, 괄호는 상위 3 개 적중 수)

첫 검색어 조건	A 키워드(BM25)	B Hybrid	C Agent
조건 ① 질문 원문	0	11 (상위 3 개 13)	16 (C-ko)
조건 ② 영어 검색어	13 (상위 3 개 15)	15 (상위 3 개 16)	16 (상위 3 개 16)

표 11. 지연과 호출 수 비교 (C 는 OpenAI 평가 실행 기준)

방식	평균 지연	질문당 Tool / LLM 호출	비고
A 키워드(BM25)	0.002–0.003 초	1.0–1.75 / 0	LLM 을 쓰지 않아 모델과 무관한 검색 지연. **[맥북의 OpenAI 평가 결과 파일에서 같은 실행의 값으로 확인]**
B Hybrid	0.17–0.25 초	4.0 / 0	위와 같음
C Agent (OpenAI, v3)	**2.57 초** (최대 6.74)	**[맥북 결과 파일에서 집계]** / 약 2.1	LLM 호출 42 회, 입력 33,753·출력 1,717 토큰

Agent(C)는 조건에 따라 B 보다 1~5 문항을 더 맞혔다. 정확도보다 확실한 차이는 **안전성**이다. 자막으로 알 수 없는 요청 2 개를 모두 거부했고, 화자 질문에는 "확인 불가"를 안내했으며, 검증을 통과한 근거만 보여 주고, 근거 없이 화자를 단정한 경우는 0 건이었다. 대신 지연과 LLM·Tool 호출이 늘어난다(4.2 절).

3.4 추가 개선 시도: 이중 첫 검색

왜 했나. 같은 의미의 질문이 표현에 따라 찾기도 하고 못 찾기도 했다(4.2 절). 원인을 보니 Router 가 만든 영어 검색어와 번역된 자막의 단어가 달랐다(예: 검색어는 "manitto", 자막은 "Secret Santa"). 반면 질문을 한국어 그대로 의미 검색에 넣으면 정답이 1 위로 나왔다.

어떻게 했나. 첫 검색에서 Router 의 영어 검색어와 질문 원문을 **둘 다** 검색하고, 두 결과를 순위대로 번갈아 채워(Router 결과 먼저) 판정할 후보 3 개를 만든다. LLM 호출과 판정 후보 수는 늘지 않고 검색만 1 번 늘어난다. 재검색 라운드, 후속 질문, 알 수 없는 요청에는 적용하지 않으며, 기본값은 꺼짐이다.

표 12. 이중 첫 검색 동작 확인 (평가셋이 아닌 직접 실행, 질문당 1 회)

질문 (실제 자막, OpenAI, 1 회 실행)	미적용 (기존)	적용 (이중 첫 검색)
마니또 "...자기 자신을 뽑은 장면"	못 찾음, 3 라운드, 7.7 초	**찾음 3:09, 1 라운드, 3.4 초**
구운 오징어 / 4 월 20 일 앨범 / 한우 축제	찾음, 1 라운드, 2.4 / 2.9 / 2.6 초	같은 시점을 찾음, 4.3 / 4.6 / 4.5 초

- **표의 질문은 점검용으로 고른 것이다**: 마니또는 기존에 못 찾던 사례이고, 오징어·4 월 20 일·한우는 이미 찾던 질문이 계속 찾아지는지 보려고 골랐다. 질문을 체계적으로 선정한 것이 아니라서 정확도 판단은 정답 평가셋 재측정으로 한다.
- **확인된 것**: 못 찾던 마니또 질문 두 표현을 1 라운드에 찾았다. 장면 질문 10 건이 모두 "찾음"이었고, 그중 오징어(45:53), 료칸(2:18), 곰(58:04), 4 월 20 일(34:46), 한우(0:39)는 사용자가 확인한 시점과 일치했다. 김치찌개는 1 위가 확인 시점(10:33)이 아닌 같은 영상의 다른 관련 장면(8:37)이었고, 적용하지 않은 실행에서도 같은 결과가 나온 적이 있어 이 기능 때문인지는 구분하지 못했다.
- **구현 중에 발견한 성능 저하**: 코드 리뷰에서 "같은 청크에서 검색어마다 고른 근거 cue 가 다른데 Router 쪽만 남긴다"는 지적을 받아 후보를 근거 cue 기준으로 합치게 고쳤다. 그런데 이때 점수를 합산해 순위를 매겼더니 실제 자막에서 오히려 정답이 밀려났다. 두 검색이 함께 고른 약한 후보가 1 위가 되어 후보 3 자리 중 하나를 차지하고, Router 의 2 위였던 정답이 밀려난 것이다("구운 오징어" 질문이 적용 전에는 4/4 번 45:53, 수정 직후에는 3/3 번 42:41). LLM 없이 같은 검색 결과로 원인을 확인한 뒤 점수 합산 대신 **순위를 번갈아 채우는 방식**으로 바꿨고, 이 경우를 테스트로 고정했다. 단위 테스트만으로는 이 문제를 찾지 못했고 실제 자막으로 확인해야 했다.
- **단점**: 검색이 두 번이라 질문당 약 1~2 초 느려진다. 질문 원문 검색에서도 약한 질문에는 도움이 없고, 원문 검색 결과가 후보 자리를 일부 차지하므로 Router 결과의 3 위 이하는 후보에서 빠진다. 판정 후보를 늘리면 해결되지만 토큰과 외부 전송량이 늘어서 택하지 않았다.
- **측정하지 못한 것**: 정답 평가셋 20 문항에서의 정확도와 틀린 장면 증가 여부. 그래서 이 기능은 **효과가 입증된 개선이 아니라 가능성을 확인한 단계**이며, 재측정 결과에 따라 켤지 철회할지 정한다.

[내일(Day 3) 맥북에서 정답 평가셋 20 문항 A/B/C 를 이중 첫 검색 미적용(v3)과 적용(v4)으로 재측정한

결과표와 실행 화면 캡처를 이 절에 삽입. 정답 평가셋 승인 파일이 맥북에만 있어 오늘은 기존 측정값(v2·v3)과 위의 직접 실행만 사용. (선택) 청크 길이 20 초/10 초를 바꿔 비교한 결과가 있으면 4.3 절의 "비교하지 않았다" 문장을 결과로 바꾼다]

4. 한계와 트레이드오프

4.1 Multi-Agent(Supervisor)로 나누지 않은 이유와 설계안

- **이유**: 일이 순서대로 짧고(LLM 호출 2~6 회) 병렬로 할 일이 없다. 도구가 3~4 개뿐이고 호출 순서가 정해져 있어 LLM 이 도구를 고를 필요도 없다. Supervisor 를 두면 "누구에게 맡길지"를 정하는 LLM 호출이 추가되어 지연, 토큰, 실패 지점이 늘어난다. 또 실제 실패 원인을 열어 보니 구조가 아니라 **검색어의 단어 차이와 판정 기준**이었다.
- **나눴을 때의 설계안**: Supervisor 가 다음에 부를 Agent 를 고르고, Search Agent 는 MCP 도구를 스스로 골라 호출하며, Verifier Agent 는 판정만 한다. 단계 수 상한, 시간 제한, 재검색 상한, 잘못된 선택 시 종료 규칙을 두고, 인용과 시각 검증은 코드로 유지한다. 환경변수로 켜서 기존 경로를 보존하고 같은 평가셋으로 비교한다.
- **도구 선택은 코드가 정한다**: 지금은 그래프 코드가 MCP 도구를 정해진 순서로 호출하고, LLM 이 도구를 고르는 방식(function calling)이 아니다. MCP 를 "자료에 안전하게 접근하는 표준 통로"로 쓰는 것이며, Search Agent 가 도구를 스스로 고르게 하는 것은 위 설계안의 한 부분이다.
- **예상되는 단점(실험으로 확인하지 못한 가설)**: LLM 호출과 지연 증가, Supervisor 가 잘못 판단하는 새로운 실패, 순환 구조에서의 무한 반복 가능성. 자율 검색이 더 많이 찾을 수 있다는 것도 가설이다. **이번 제출에서는 구현하지도 측정하지도 않았다.**

4.2 개선이 만든 약점과 정확도 한계

- **토큰 증가**: v3 는 호출 수가 줄었지만 프롬프트에 프로필과 규칙을 추가해 입력 토큰이 약 24% 늘었다(56,941 → 70,770). v3 만의 비용은 따로 확인하지 못했고, LangSmith 합계 $0.30 에서 v2 $0.1466 을 뺀 약 $0.15 로 추정한다(반올림 오차가 있다).
- **Agent 가 더 낫다는 증거가 약하다**: Agent(16/18)와 B(15/18 또는 11/18)의 차이는 1~5 문항이고 질문당 1 회만 실행했다. 같은 설정(temperature 0, seed 42)으로 두 번 실행했을 때도 한 문항이 달랐다(17/18 → 16/18). 1 문항 차이는 우연일 수 있다.
- **지연과 호출 증가**: B 는 평균 0.2 초대인데 C 는 OpenAI 에서 평균 2.6 초(최대 6.7 초)이다. 질문당 LLM 호출이 약 2 회이고 재검색이 있으면 4 회로 늘어 토큰(비용)도 함께 늘어난다.

- **틀린 장면을 "찾음"으로 답함**: 의미 검증은 근거를 꾸며내는 것은 막지만 장면이 맞는지까지는 보장하지 못한다. 같은 말을 하는 장면이 여럿인 질문에서 비슷한 다른 장면을 정답으로 낸 사례가 있다. 평가셋이 장면마다 정답을 하나만 두어서 이런 경우는 오답으로 집계된다.
- **번역 자막의 단어 차이와 결과의 변동**: 질문의 단어와 번역된 자막의 단어가 다르면 후보에 오르지 않는다. 마니또 질문은 표현에 따라 찾기도 하고("자기 자신을 뽑은") 못 찾기도 했다("자기 자신 뽑은"). 이를 줄이려고 이중 첫 검색을 시도했다(3.4 절).
- **동음이의어**: "감"(과일/느낌)처럼 Router 가 한 뜻만 골라 검색하면 다른 뜻의 장면이 후보에 오르지 않는다. 여러 뜻을 모두 검색하게 하는 방식은 gpt-4o-mini 가 따라 주지 않아 폐기했다. Router 를 gpt-4o 로 바꾸면 해결되는 사례를 확인했지만 토큰 비용 때문에 기본값은 gpt-4o-mini 로 두었다(평가셋 전체에 미치는 영향은 측정하지 못했다).

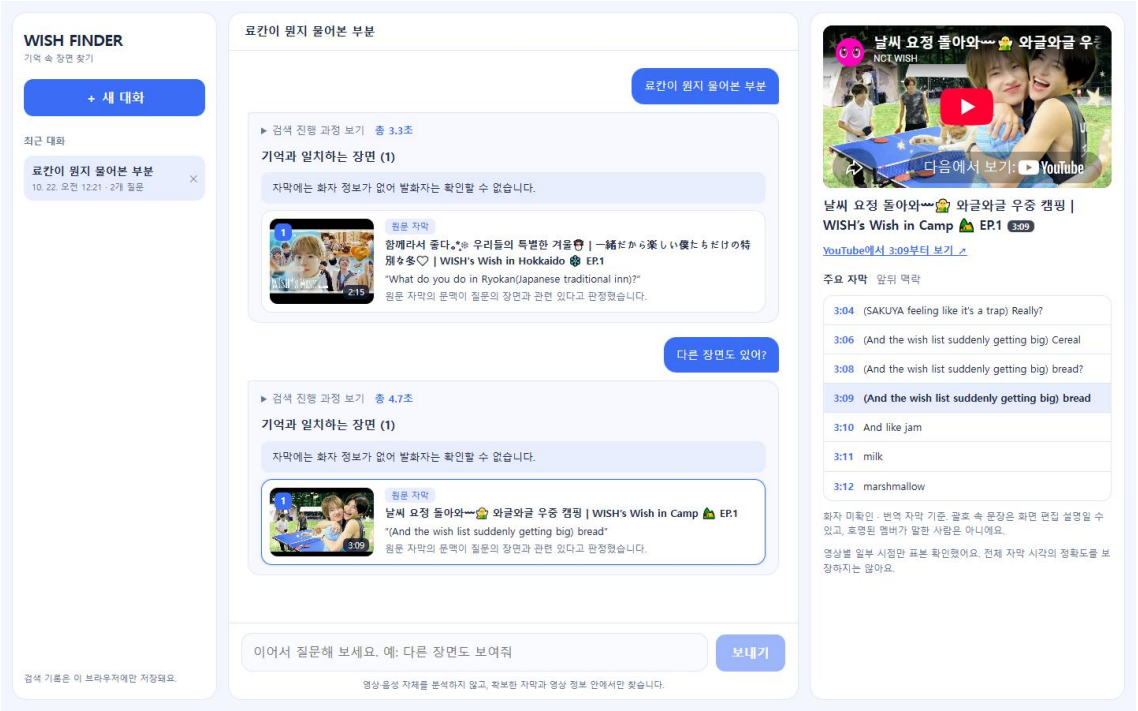


그림 9. 후속 질문의 실패 사례 (이중 첫 검색을 끈 채 실행 중이던 서버). 대상 없는 "다른 장면도 있어?"에 료칸과 무관한 캠핑 영상의 "bread" 장면을 정답으로 냈다(이 서버에서 같은 입력으로 6/6 번 재현). 주제를 말한 후속 질문은 정상이다.

- **후속 질문**: 위 사례처럼 "다른 장면도?"처럼 지시어만 있는 후속 질문에서 엉뚱한 장면이 나왔다. 다만 같은 입력이 최신 코드(이중 첫 검색 적용)로 띄운 서버에서는 3/3 번 모두 료칸 관련 장면(48:11)이 나와 **재현되지 않았다.** 이중 첫 검색은 후속 질문에는 적용하지 않는 설계라 이 차이의 원인은 알지 못하며, 좋아졌다고 주장하지 않는다.
- 코드 리뷰에서 "Router 가 다시 쓴 질문이 검증 기준까지 바꾸면 틀린 재작성이 스스로 정답으로 확정되는 문제"를 지적받아, 검증 기준은 원래 질문과 직전 대화로 고정하고 다시 쓴 질문은 검색에만

쓰도록 나눴다. 다만 이 규칙은 프롬프트로 지시하는 것이지 코드로 강제하는 검사가 아니다. 후속 질문의 정확도는 평가셋으로 측정하지 않았다.

- 이미 보여 준 장면을 제외할 때 모든 후보의 원문을 조회하던 방식을 고쳐, 제외한 뒤 남은 최대 3 개만 조회하도록 했다(조회 횟수가 결과 수에 비례).

4.3 데이터·환경·운영 한계

- 영상과 음성은 분석하지 않는다. 입력은 영어 번역 자막과 일부 영상 정보뿐이고 화자는 알 수 없다. 정답 시각은 영상마다 3 곳을 표본 확인한 것이며 전수 검증이 아니다.
- 평가 표본이 작다(장면 18 문항, 질문당 1 회 실행). p95 지연, 반복 실행의 분산, 못 찾아야 하는 질문의 정확도(확정된 문항 없음), 사람이 판정한 의미 품질, 다른 LLM 은 측정하지 못했다. 개선은 평가셋의 실패 사례를 본 뒤 한 것이라 결과가 낙관적일 수 있고, 별도의 확인용 질문으로 검증하지 못했다.
- 실행 순서가 고정이라 앞선 실행이 만든 캐시가 뒤 조건의 지연을 줄였을 수 있다. 청크 길이(20 초, 10 초 겹침)와 RRF 상수(60)도 다른 값과 비교하지 않았다.
- 키워드 검색(BM25)은 공백으로 단어를 나누므로 일본어·중국어처럼 띄어쓰기가 없는 언어에서는 사실상 동작하지 않는다(영어 외 언어는 측정하지 않았다). 자막 형식은 SRT 만 지원하고 한 영상의 다국어 자막은 다루지 않는다.
- **첫 기동**: 검색 인덱스를 처음 만드는 데 CPU 에서 약 200 초가 걸려 MCP 초기화 제한(180 초)을 넘기므로 미리 만들어 두어야 했다. Windows 에서는 MCP 자식 프로세스가 가끔 시작에 실패했고, UTF-8 설정을 넘기고 오류 출력을 파일로 돌려 대응했다(원인을 완전히 밝히지는 못했다).
- **LangSmith 에서 MCP 도구 호출이 보이지 않는다**: 그래프 단계와 LLM 호출만 올라가고 MCP 도구 호출은 따로 잡히지 않는다(개요 화면의 Tools 패널이 "No tool calls detected"). 도구 호출 수는 로컬 로그의 tool_calls 로만 확인할 수 있다.
- OpenAI 를 쓰면 질문과 후보 자막 일부(최대 3 구간, 앞뒤 20 초)가 외부로 전송된다. LangSmith 도 같은 내용을 올리므로 비공개 프로젝트에서만 쓴다.

5. 느낀점과 이후 개선 포인트

5.1 느낀점

[이 절은 작성자 본인의 경험으로 고쳐 쓰세요. 아래는 이 프로젝트에서 실제로 있었던 일을 바탕으로 쓴 초안입니다. 가장 막혔던 순간, 예상과 달랐던 결과, 다시 한다면 바꿀 점을 떠올리면 쓰기 쉽습니다.]

- **문제는 구조보다 데이터에 있었다.** 검증과 재검색을 하는 Agent 를 만들면 정확도가 오를 줄 알았다. 그런데 못 찾은 질문을 하나씩 열어 보니 원인은 번역 자막의 단어였다. 질문은 "마니또"인데 자막에는

"Secret Santa"로 적혀 있었다. 프롬프트와 설정을 고치자 Agent 가 없는 키워드 검색(A)도 5 에서 13 으로 올랐다.

- **측정해야 개선이라고 말할 수 있다.** 같은 설정으로 두 번 실행해도 한 문항이 달랐고, 개선한 버전(v3)은 정답 수가 그대로인데 입력 토큰은 24% 늘었다. "좋아졌다"고만 쓰면 틀리는 경우가 있어서, 무엇이 얼마나 달라졌는지 숫자로 적는 습관이 생겼다.
- **테스트가 통과해도 실제 데이터로 봐야 한다.** 이중 첫 검색을 고치다가 단위 테스트는 모두 통과했는데, 실제 자막으로 실행해 보니 정답이 후보에서 밀려나 있었다. 원인을 찾고 나서야 방식을 바꿀 수 있었다.
- **할 수 없는 것을 알려 주는 것도 기능이다.** 자막에는 말한 사람이 없어서 화자는 "확인 불가"로 안내하고, 인용과 시각은 LLM 이 쓰지 않고 코드가 원본에서 만들게 했다. 그 결과 근거 없이 화자를 단정한 경우가 0 건이었다. 반대로 후속 질문처럼 아직 틀리는 부분은 숨기지 않고 한계로 적었다.
- **막혔을 때는 추측보다 기록이다.** Windows 에서 MCP 서버가 가끔 시작에 실패했을 때, 짐작으로 고친 것은 두 번 빗나갔고 자식 프로세스의 출력을 파일로 남긴 뒤에야 원인을 좁힐 수 있었다.

5.2 이후 개선 포인트

표 13. 이후 개선 포인트

영역	개선 내용	확인 방법
구조	Supervisor 가 다음에 부를 Agent 를 고르고 Search Agent 가 MCP 도구를 스스로 고르는 구조를 환경변수로 켤 수 있게 만든다(4.1 절).	같은 정답 평가셋으로 정확도·지연·호출 수·토큰·재검색 횟수 비교
검색 품질	청크 길이(지금 20 초, 10 초 겹침)와 RRF 상수(60)를 바꿔 비교한다. 청크 길이는 설정과 실행 스크립트가 준비되어 있다.	정답 평가셋에서 1 위 적중 수 비교(설정별 A/B 평가)
단어 차이	이중 첫 검색의 재측정과 기본값 결정, 못 찾았을 때만 "혹시 이런 뜻인가요?"라고 되묻기, Router 모델과 동의어 사전 개선.	못 찾은 질문의 감소, 틀린 장면을 "찾음"으로 답한 수가 늘지 않는지 확인
후속 질문	다시 쓴 질문이 원래 맥락을 벗어나지 않았는지 코드로 점검하고, 후속 질문 전용 평가셋을 만든다.	후속 질문에서 엉뚱한 장면이 나오는 비율
사용성	검색 중에도 새 대화를 시작할 수 있게 한다(진행 중인 검색 취소 또는 대기열). 같은 영상에서 영상별 시점을 더 많이 보여 주는 확장도 고려한다.	검색 중 새 대화 가능 여부, 판정 후보를 늘렸을 때 토큰·지연 증가량
관측	LangSmith 에 MCP 도구 호출도 기록한다. 화면용 조회(앞뒤 자막, 영상 정보)를 MCP 도구에서 API 로 옮겨 도구 구성을 정리한다.	LangSmith 트레이스에서 도구 호출 확인
평가	반복 실행으로 결과의 흔들림을 재고, 확인용 질문을 따로 두고, 못 찾아야 하는 질문과 비슷한 장면이 여럿인 질문의 정답을 보완한다.	반복 실행 간 편차, 못 찾아야 하는 질문의 정확도

영역	개선 내용	확인 방법
대상 확장	영상마다 자막 언어를 설정하고, 다른 대상의 프로필과 자막 형식(SRT 외)을 지원한다. 띄어쓰기 없는 언어를 위한 단어 분리를 넣는다.	다른 대상·언어에서의 소규모 평가셋

부록 A. 실행 화면 모음

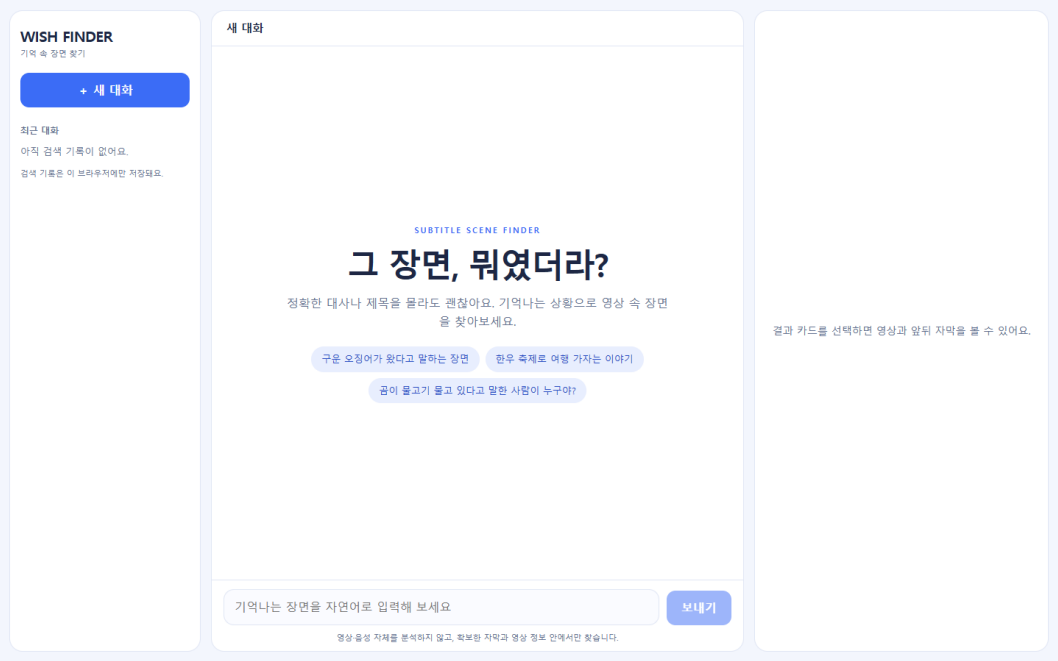


그림 10. 첫 화면. 예시 질문 3 개가 있고, 검색 기록은 이 브라우저에만 저장된다.

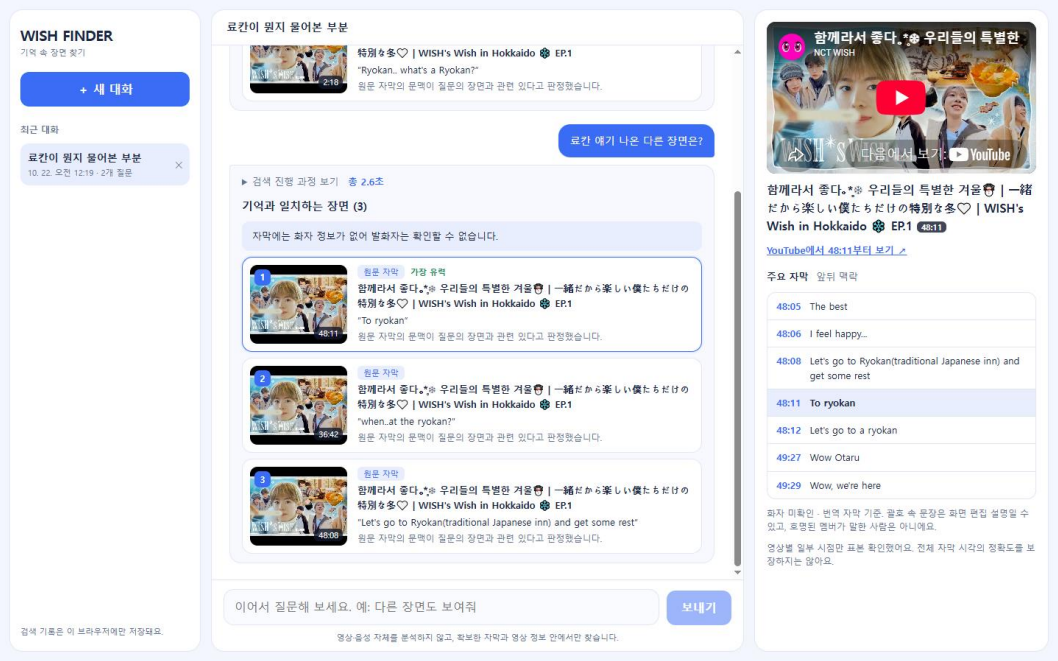


그림 11. 후속 질문("료칸 얘기 나온 다른 장면은?"). 직전 질문과 이미 보여 준 장면을 함께 보내 다른 장면을 찾는다. 서버는 대화를 저장하지 않는다.

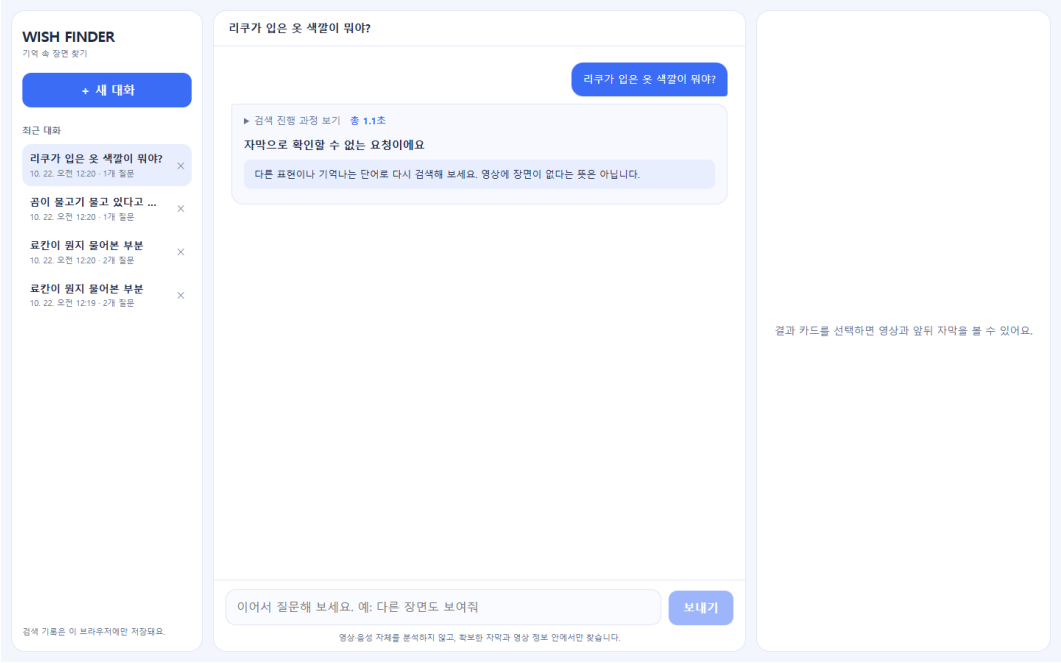


그림 12. 자막으로 알 수 없는 질문("...입은 옷 색깔이 뭐야?"). 검색 없이 1.1 초 만에 "자막으로 확인할 수 없는 요청"으로 종료한다.

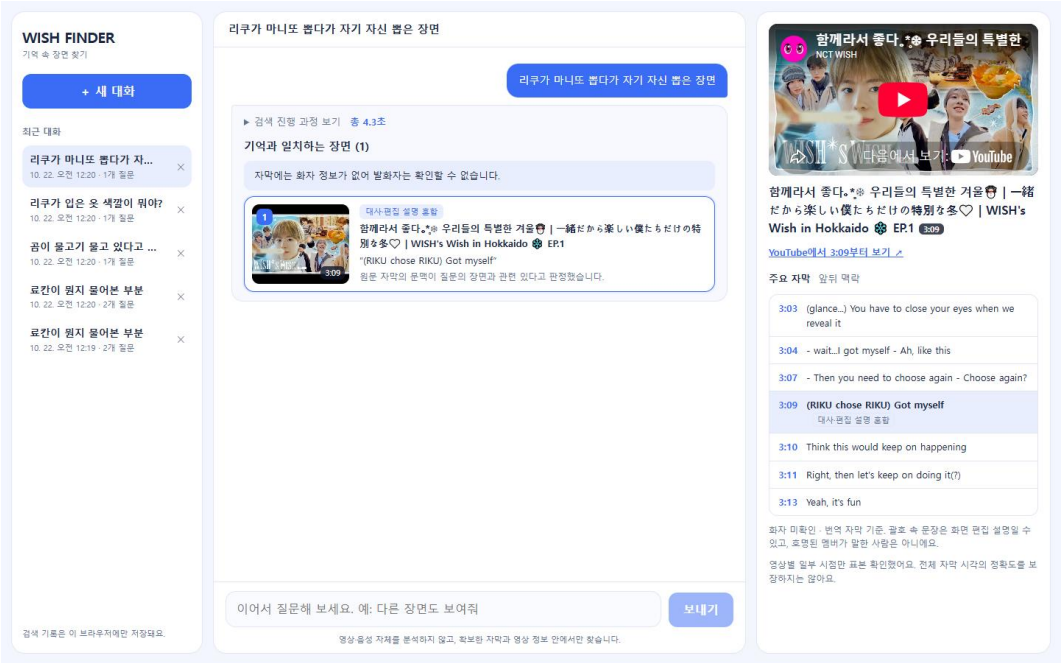


그림 13. 마니또 질문("...자기 자신 뽑은 장면"). 자막의 화면 편집 설명 "(RIKU chose RIKU)", 3:09 시점을 1 라운드에 찾았다(총 4.3 초, 이중 첫 검색 적용). 적용하지 않으면 못 찾았다(3.4 절).

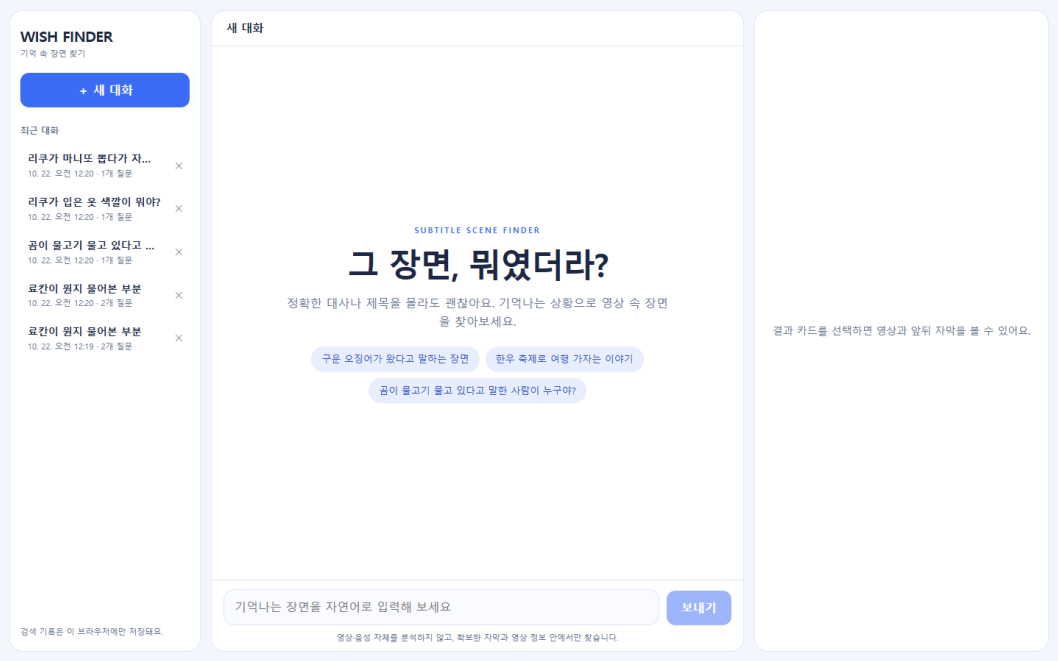


그림 14. 새로고침 후 검색 기록 복원. 저장된 결과는 다시 검색하지 않고 그대로 보여 준다.

부록 B. 실행·재현 방법

표 14. 실행·재현

항목	명령 / 설명
설치	py -3.11 -m venv .venv → .venv₩Scripts₩python.exe -m pip install -r backend₩requirements.txt → npm --prefix frontend ci
백엔드 (OpenAI)	scripts/run_backend.sh openai (Windows 는 Git Bash 또는 scripts₩run_backend.ps1). .env 에 OPENAI_API_KEY 가 필요하고 과금 확인(y/N)이 나온다.
프런트엔드	scripts/run_frontend.sh → http://localhost:5173/ (127.0.0.1 이 아닌 localhost 로 접속)
테스트	.venv/Scripts/python.exe -m pytest backend/tests -q / npm --prefix frontend run test -- --run
평가	python -m eval.run_eval --dataset eval/private/gold.jsonl --systems A,B,C --control --max-llm-calls 150 --output eval/private/<결과>.json
공개 범위	원본 자막, 정답 평가셋, 질문별 결과, 검색 인덱스, .env 는 Git 에 올리지 않는다(eval/private/, data/raw/). 제삼자는 같은 자막 결과를 재현할 수 없고, 합성 자막 기반 테스트로 경로와 지표를 확인한다.

부록 C. 설계 결정 기록 (ADR)

표 15. ADR 목록 (docs/adr/)

번호	주제
0001	자막 정보와 화자 미식별 처리

번호	주제
0002	Hybrid 검색과 다국어 질의
0003	LangGraph 역할 분리와 재검색 상한 (독립 Multi-Agent 로 과장하지 않음)
0004	공식 MCP SDK 2.x MCPServer 와 로컬 stdio 경계
0005	Vue + FastAPI POST SSE
0006	원본 cue 보존과 시간 창 근거 검증
0007	결과 카드의 YouTube 임베드와 오류 폴백
0008	LLM 백엔드 환경변수 교체 (Ollama 개발 / OpenAI 제출)
0009	대상 프로필, 번역 자막 프롬프트, 중복 재검색 제거
0010	대화형 3 단 화면, 브라우저 검색 기록, 후속 질문
0011	첫 검색에서 Router 검색어와 질문 원문을 함께 검색 (이중 첫 검색, 기본 꺼짐)

부록 D. 정답 평가셋 (gold)

평가에 쓴 정답 평가셋 20 문항의 구성은 3.1 절과 같다(장면 질문 18: 일반 13, 같은 말을 하는 장면이 여러 개인 질문 1, 화자 질문 4 / 자막으로 알 수 없는 요청 2). 질문과 정답 시점은 공개 저장소에 올리지 않는 파일에 있어 이 PC 에서는 확인할 수 없다.

[맥북에서 정답 평가셋(eval/private/gold.jsonl)의 문항 번호, 유형, 질문, 정답 영상과 시점(허용 구간)을 표로 삽입. 질문과 정답 시점을 제출 PDF 에 공개할지는 본인이 정한다]